

Automated High-Throughput Banking & Real-Time Fraud Detection Engine

Course: CS212 Object-Oriented Programming

Institution: NUST School of Electrical Engineering & Computer Science (SEECs)

Authors: Danyal Aqeel (Lead), Hamza Zahoor (Junior), Muhammad Arslan (Senior)

Date: August 03, 2026

Live Repository: <https://github.com/DANYALAQEEL/CS212-Banking-Fraud-Engine>

1. Executive Summary

This project delivers a multi-threaded, real-time banking execution system integrated with an automated fraud security engine and interactive JavaFX telemetry dashboard. Built in Java 17 LTS, the system guarantees zero thread deadlocks during concurrent multi-account balance transfers, enforces real-time fraud rules via sliding-window sliding algorithms, and provides multi-format file persistence.

2. Object-Oriented Programming (OOP) Principles Applied

A. Encapsulation & Thread Visibility

- All core account balances are encapsulated with private fields and guarded by explicit fair `ReentrantLock(true)` instances.
- The `balance` field is declared `volatile` to guarantee immediate memory visibility across multi-core CPU hardware threads without requiring blocking reads.

B. Polymorphism & Inheritance Hierarchy

- `Account` serves as an abstract base class with a polymorphic `debit(double amount)` method contract.
- Subclasses enforce distinct business rules:
 1. `SavingsAccount`: Enforces minimum balance requirements (\$RS 1,000.0\$) and applies compound interest.
 2. `CheckingAccount`: Supports overdraft protection limits up to \$RS 5,000.0\$ and applies per-transaction service fees.
 3. `CreditAccount`: Represents revolving credit limits, allowing debits up to the credit limit while tracking APR interest.

C. Abstraction & Interface Contracts

- Interfaces such as `LockStrategy` and `JavaFXEventListener` abstract lock acquisition strategies and UI thread handoff callbacks from concrete execution logic.

3. Concurrency Engine & Deadlock Prevention Mechanics

Coffman's Condition Elimination (Zero Deadlocks)

Standard multi-account transfers lock two accounts simultaneously (source and destination). Naive implementations that lock accounts in order of argument arrival suffer from Coffman's Circular Wait condition.

To prevent circular wait, `AccountLockManager.java` sorts account locks deterministically by Account ID (`acc1.compareTo(acc2)`) before lock acquisition:

```
Account firstLock = accA.compareTo(accB) < 0 ? accA : accB;
Account secondLock = accA.compareTo(accB) < 0 ? accB : accA;
firstLock.getLock().lock();
secondLock.getLock().lock();
```

This guarantees strict total order lock acquisition across all worker threads, completely eliminating deadlocks.

4. Real-Time Fraud Detection Engine Algorithms

`FraudDetectionEngine.java` evaluates every transaction against four security rules:

1. **Large Amount Anomaly Rule:** Flags transfers exceeding \$RS 100,000.0\$.
2. **High Velocity Rule:** Quarantines accounts attempting \$> 5\$ transfers within a sliding 10-second window.
3. **Suspicious Target Rule:** Detects rapid transfers to high-risk destination accounts.
4. **Rapid Drain Rule:** Quarantines accounts losing \$> 80\%\$ of liquidity within a 1-minute window.

5. Persistence Strategy & Storage Architecture

The system provides multi-format data storage:

1. **CSV Ledger File Persistence** (`LedgerFileManager.java`): Exports and imports account balances, preserving polymorphic subclass properties.
2. **JSON Audit Streams:** Formats transaction settlement logs into JSON objects for compliance auditing.
3. **Java Binary Snapshot Serializer** (`BinaryStateSerializer.java`): Saves complete `EngineSnapshot` objects using `ObjectOutputStream`. Marked `ReentrantLock` as transient with custom `readObject` lock re-initialization upon deserialization.

6. Empirical Test Suite & Verification Matrix

The project includes 8 comprehensive JUnit 5 test suites with **44 passing unit tests**:

| Test Suite File | Focus Area | Result |

| :--- | :--- | :---: |

| `AccountTest.java` | Subclass business rules & balance validation | **PASS (9/9)** |

| `TransferEngineTest.java` | Multi-account atomic transfers & TOCTOU re-validation | **PASS (4/4)** |

| `TransactionPipelineQueueTest.java` | Priority queue bounded capacity & condition backpressure | **PASS (4/4)** |

| `FraudDetectionEngineTest.java` | Sliding window fraud rules & risk scoring | **PASS (8/8)** |

| `LockStrategyTest.java` | Deadlock prevention vs naive deadlock detection | **PASS (10/10)** |

| `DashboardControllerTest.java` | UI thread handoff event bridge & account seeding | **PASS (2/2)** |

TelemetryViewControllerTest.java	TPS throughput & composite threat level scoring	**PASS (4/4)**
PersistenceTest.java	CSV/JSON persistence & binary snapshot roundtrips	**PASS (3/3)**
TOTAL	Full System Verification	44/44 PASS

7. Team Contribution Matrix

- **Danyal Aqeel (Lead):** Project architecture, core concurrency engine (AccountLockManager, TransferEngine, TransactionPipelineQueue, FraudDetectionEngine), deadlock testing, and repository management.
- **Hamza Zahoor (Junior):** Domain model hierarchy (Account, SavingsAccount, CheckingAccount, CreditAccount), UML diagrams v1.0, and File Persistence & Binary State Serialization (LedgerFileManager, BinaryStateSerializer).
- **Muhammad Arslan (Senior):** JavaFX GUI layout (MainApp, main_dashboard.fxml, styles.css), Controllers (DashboardController, TelemetryViewController), Thread Handoff Bridge (JavaFXEventListener), and Evolved UML v2.0.